

TASK 02

Step 1 — Create User Entity

Field	Purpose
id	Unique ID for each user
name	User's name
email	User's login/email address
password	Encrypted user password
role	User's authorization role, e.g. USER, ADMIN
status	Account status, e.g. ACTIVE, INACTIVE
createdAt	Date and time when the account was created

Example Entity

```
package com.example.demo.entity;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
@Data

public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    @Column(unique = true, nullable = false)

    private String email;
```

```
private String password;
private String role;
private String status;
private LocalDateTime createdAt;
}
```

in the database

register a user, a record can look like:

id : 1

name : Madhav

email : madhav@gmail.com

password: encrypted-password

role : USER

status : ACTIVE

createdAt: 2026-09-17 09:30

Never store the user's actual password as plain text. Use a password encoder such as BCrypt.

Step 2 — Create Registration DTO

DTO means **Data Transfer Object**.

RegisterRequest is used to receive the information that comes from the frontend during registration.

Example RegisterRequest

```
package com.example.demo.dto;
import lombok.Data;
@Data
public class RegisterRequest {
    private String name;
    private String email;
    private String password;
}
```

The frontend sends:

```
{  
  "name": "Madhav",  
  "email": "madhav@gmail.com",  
  "password": "Password@123"  
}
```

Spring Boot converts this JSON into a RegisterRequest object.

Complete Registration Flow

Frontend

| POST /api/auth/register

v

AuthController

v

RegisterRequest DTO

v

AuthService

|-- Check email

|-- Encrypt password

|-- Set role

|-- Set status

|-- Set createdAt

v

User Entity

v

UserRepository

v

Database

Example request

POST /api/auth/register

Content-Type: application/json

```
{  
  "name": "Madhav",  
  "email": "madhav@gmail.com",  
  "password": "Password@123"  
}
```

Backend processing

The backend should approximately do:

```
User user = new User();  
user.setName(request.getName());  
user.setEmail(request.getEmail());  
user.setPassword(passwordEncoder.encode(request.getPassword()));  
user.setRole("USER");  
user.setStatus("ACTIVE");  
user.setCreatedAt(LocalDateTime.now());  
userRepository.save(user);
```

Notice this important line:

```
passwordEncoder.encode(request.getPassword())
```

Instead of storing:

```
Password@123
```

the database stores a BCrypt hash similar to:

```
$2a$10$....
```

Step 3 — Validate Registration

Before creating a user in the database, the application should check whether the registration data is correct.

Name is required

The user should not be allowed to register without a name.

@NotBlank(message = "Name is required")

private String name;

Example:

Invalid:

```
{  
  "name": "",  
  "email": "madhav@gmail.com",  
  "password": "Password@123"  
}
```

Valid:

```
{  
  "name": "Madhav",  
  "email": "madhav@gmail.com",  
  "password": "Password@123"  
}
```

Email must be valid

The email should follow a valid email format.

@Email(message = "Invalid email")

@NotBlank(message = "Email is required")

private String email;

Invalid:

madhav@gmail

Invalid:

madhav.com

Valid:

madhav@gmail.com

Password is required

The password cannot be empty.

```
@NotBlank(message = "Password is required")
```

```
private String password;
```

Invalid:

```
{  
  "password": ""  
}
```

Valid:

```
{  
  "password": "Password@123"  
}
```

Password must meet minimum requirements

we can specify rules such as:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character

Example:

```
@Size(min = 8, message = "Password must contain at least 8 characters")
```

```
private String password;
```

we can also use `@Pattern` for stronger rules.

Example:

```

@Pattern(
    regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&]).{8,}$",
    message = "Password must contain uppercase, lowercase, number and special character"
)

private String password;

```

Complete RegisterRequest

You can combine these validations:

```

package com.example.demo.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public class RegisterRequest {

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email")
    private String email;

    @NotBlank(message = "Password is required")
    @Pattern(
        regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&]).{8,}$",
        message = "Password must contain at least 8 characters, uppercase, lowercase, number
and special character"
    )
    private String password;

    // Getters and setters
}

```

Using Validation in Controller

we need @Valid in your controller.

```
@PostMapping("/register")
public ResponseEntity<String> register(
    @Valid @RequestBody RegisterRequest request) {
    // Registration logic
    return ResponseEntity.ok("User registered successfully");
}
```

Here: @Valid

Spring, check all the validation annotations before processing this request.

Step 4 — Hash Passwords

Hashing converts the original password into a one-way representation.

Example:

Password@123

↓

BCrypt

↓

\$2a\$10\$....

The database stores the BCrypt hash instead of the original password.

Configure BCryptPasswordEncoder

Create a configuration class:

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
```

@Configuration

```
public class PasswordConfig {  
    @Bean  
    public PasswordEncoder passwordEncoder() {  
        return new BCryptPasswordEncoder();  
    }  
}
```

Now Spring can provide PasswordEncoder wherever you need it.

Hash the Password During Registration

In your service:

@Service

```
public class AuthService {  
    private final PasswordEncoder passwordEncoder;  
    private final UserRepository userRepository;  
    public AuthService(  
        PasswordEncoder passwordEncoder,  
        UserRepository userRepository) {  
        this.passwordEncoder = passwordEncoder;  
        this.userRepository = userRepository;  
    }  
    public void register(RegisterRequest request) {  
        User user = new User();  
        user.setName(request.getName());  
        user.setEmail(request.getEmail());  
        // Hash password  
        user.setPassword(  

```

```

        passwordEncoder.encode(request.getPassword())
    );
    user.setRole("USER");
    user.setStatus("ACTIVE");
    user.setCreatedAt(LocalDateTime.now());
    userRepository.save(user);
}
}

```

The important line is:

```

user.setPassword(
    passwordEncoder.encode(request.getPassword())
);

```

Step 5 — Create Registration API

POST is used when we want to **create new data**.

we are creating a new user account.

Example request

```

{
  "name": "John",
  "email": "john@example.com",
  "password": "Password@123"
}

```

The request goes to:

POST /api/v1/auth/register

Controller

Create an AuthController:

```
@RestController
```

```

@RequestMapping("/api/v1/auth")

public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {

        this.authService = authService;
    }

    @PostMapping("/register")

    public ResponseEntity<String> register(

        @Valid @RequestBody RegisterRequest request) {

        authService.register(request);

        return ResponseEntity

            .status(HttpStatus.CREATED)

            .body("User registered successfully");

    }

}

```

Understanding

`@RequestMapping("/api/v1/auth")`

Creates the common URL: `/api/v1/auth`

Then: `@PostMapping("/register")`

adds: `/register`

So the complete endpoint becomes:

POST `/api/v1/auth/register`

Step 6 — Check Duplicate Users

suppose the database already contains:

id: 1

name: John

email: john@example.com

Someone tries to register again:

```
{  
  "name": "John Smith",  
  "email": "john@example.com",  
  "password": "Password@123"  
}
```

The backend should detect that the email already exists.

Create UserRepository

@Repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    boolean existsByEmail(String email);  
    Optional<User> findByEmail(String email);  
}
```

Now we can check:

```
userRepository.existsByEmail(request.getEmail())
```

Return 409 Conflict

In the service:

```
public void register(RegisterRequest request) {  
    if (userRepository.existsByEmail(request.getEmail())) {  
        throw new ResponseStatusException(  
            HttpStatus.CONFLICT,  
            "Email already registered"  
        );  
    }  
  
    User user = new User();
```

```
user.setName(request.getName());
user.setEmail(request.getEmail());
user.setPassword(
    passwordEncoder.encode(request.getPassword())
);
user.setRole("USER");
user.setStatus("ACTIVE");
user.setCreatedAt(LocalDateTime.now());
userRepository.save(user);
}
```

If the email already exists:

HTTP 409 Conflict

Example response:

```
{
  "status": 409,
  "message": "Email already registered"
}
```

409:

409 Conflict means:

The request is valid, but it conflicts with the current state of the server.

Here, the conflict is:

New registration

+

Existing email

↓

409 Conflict

Step 7 — Create Login API

POST /api/v1/auth/login

This endpoint is used when an existing user wants to **log in**.

For example:

```
{  
  "email": "john@example.com",  
  "password": "Password@123"  
}
```

The request is:

POST /api/v1/auth/login

Create LoginRequest DTO

Create:

```
public class LoginRequest {  
    private String email;  
    private String password;  
    // getters and setters  
}
```

Or with Lombok:

@Data

```
public class LoginRequest {  
    private String email;  
    private String password;  
}
```

Add Login Method to Controller

@PostMapping("/login")

```
public ResponseEntity<String> login(
```

```
        @RequestBody LoginRequest request) {  
    authService.login(request);  
    return ResponseEntity.ok("Login successful");  
}
```

So your controller now has:

```
@RestController  
@RequestMapping("/api/v1/auth")  
public class AuthController {  
    private final AuthService authService;  
    public AuthController(AuthService authService) {  
        this.authService = authService;  
    }  
    @PostMapping("/register")  
    public ResponseEntity<String> register(  
        @Valid @RequestBody RegisterRequest request) {  
        authService.register(request);  
        return ResponseEntity  
            .status(HttpStatus.CREATED)  
            .body("User registered successfully");  
    }  
    @PostMapping("/login")  
    public ResponseEntity<String> login(  
        @RequestBody LoginRequest request) {  
        authService.login(request);  
        return ResponseEntity.ok("Login successful");  
    }  
}
```

Step 8 — Verify Credentials

User provides email and password

Example:

```
{  
  "email": "john@example.com",  
  "password": "Password@123"  
}
```

The request is sent to:

POST /api/v1/auth/login

Find User

The backend searches the database using the email.

```
Optional<User> user =  
    userRepository.findByEmail(request.getEmail());
```

For example:

john@example.com

↓

Database

↓

User found

If the email doesn't exist, authentication should fail.

Compare Password

The password stored in the database should be a **BCrypt hash**, not the original password.

For example:

User enters:

Password@123

Database:

\$2a\$10\$.....

You use:

```
boolean matches = passwordEncoder.matches(  
    request.getPassword(),  
    user.getPassword()  
);
```

Important

You should **not** do this:

```
request.getPassword().equals(user.getPassword())
```

because the database contains a hash.

Instead use:

```
passwordEncoder.matches(  
    rawPassword,  
    hashedPassword  
);
```

Complete Login Service

A simple implementation is:

@Service

```
public class AuthService {  
    private final UserRepository userRepository;  
    private final PasswordEncoder passwordEncoder;  
    public AuthService(  
        UserRepository userRepository,  
        PasswordEncoder passwordEncoder) {  
        this.userRepository = userRepository;  
        this.passwordEncoder = passwordEncoder;  
    }  
    public void login(LoginRequest request) {  
        User user = userRepository  
            .findByEmail(request.getEmail())
```

```

        .orElseThrow(() ->
            new ResponseStatusException(
                HttpStatus.UNAUTHORIZED,
                "Invalid email or password"
            ));
    boolean passwordMatches =
        passwordEncoder.matches(
            request.getPassword(),
            user.getPassword()
        );
    if (!passwordMatches) {
        throw new ResponseStatusException(
            HttpStatus.UNAUTHORIZED,
            "Invalid email or password"
        );
    }
    // Authentication successful
}
}

```

Step 9 — Test Incorrect Credentials

401 Unauthorized

This means we should test what happens when the user provides incorrect login credentials.

Example

Suppose the correct password is:

Password@123

But the user sends:

```

{
    "email": "john@example.com",

```

```
"password": "WrongPassword"
}
```

The backend does:

Find email



User found



Compare passwords



Password doesn't match



401 Unauthorized

Response: 401 Unauthorized

For example:

```
{
  "status": 401,
  "message": "Invalid email or password"
}
```

401:

401 Unauthorized means the request does not have valid authentication credentials.

For login, it commonly means:

- Email doesn't exist, or
- Password is incorrect.

For security, it's generally better not to tell an attacker which of the two was wrong. Use a generic message such as:

Invalid email or password

Testing in Postman

You can test this using Postman.

Test 1 — Correct credentials

Method:

POST

URL:

http://localhost:8080/api/v1/auth/login

Body → raw → JSON:

```
{  
  "email": "john@example.com",  
  "password": "Password@123"  
}
```

Expected result:

200 OK

or, in a JWT-based implementation, a successful response containing a token.

Test 2 — Wrong password

```
{  
  "email": "john@example.com",  
  "password": "WrongPassword"  
}
```

Expected:

401 Unauthorized

Test 3 — Unknown email

```
{  
  "email": "unknown@example.com",  
  "password": "Password@123"  
}
```

Expected: 401 Unauthorized

Step 10 — Test Database

After registration, open your database and check the users table.

Suppose we registered:

Email: john@example.com

Password: Password@123

Wrong database value

password

Password@123

This means the application stored the plain-text password.

Correct database value

password

\$2a\$10\$xx

The exact hash will be different each time.

BCrypt:

BCrypt is designed specifically for password hashing and is intentionally computationally expensive, making large-scale password guessing more difficult.

Example:

@Bean

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

During registration:

```
String hashedPassword =  
    passwordEncoder.encode(request.getPassword());
```

During login:

```
passwordEncoder.matches(  
    request.getPassword(),
```

```
user.getPassword()  
);
```

Remember:

